

Advanced Software Development Testing



Simon H. Muchinenyika

Testing

Table of Contents

- Bugs
 - Causes of Bugs, Mitigating Bugs, Debugging
- Logging
 - Why Logging, What to Log, When to Log, How to Log
- Testing
 - Unit, Functional, Test Pyramid
- Documentation
 - Self-documenting, Documentation

Testing

Bugs

- software code that causes a crash or wrong output.
- sources of bugs are human errors in:
 - understanding requirements.
 - product design.
 - coding.
- stable releases have few bugs, unlike the unstable ones, which are difficult to interact with.

Testing

Mitigating Bugs

- Understand program requirements.
- Use decomposition. A good software architecture reduces program complexity.
- Write easy-to-read code.
- Consider a wide range of test cases, including boundary values.
- Write automated tests.

Testing

Debugging

- help analyse the code to understand what it does.
- logs are used for analysis.
- Use the debugger to control program flow and see current values.
- two ways to suspend program running when using the debugger:
 1. set breakpoints.
 2. press hot keys to break program execution.
- a breakpoint is a deliberate interruption of program execution to examine the program state.

Testing

Logging

- recording the execution of an app.
- such a record of an event in a software app makes it easier to understand what happened.
- examples include a timestamp, a description, and a severity level.
- logging reduces troubleshooting time.
- also makes monitoring possible, which is needed for verification and reporting.

Testing

When to Log

- Logging is essential for troubleshooting, auditing, profiling, and statistics.
- avoid excessive logging. **Why?**
- logs should cover server issues, database issues, networking issues, unanticipated user input errors, dynamically created object states, and configuration values.
- provide context when logging, e.g., adding a thread name for programs in a concurrent environment.

Testing

Logging Levels

- **Debug** logs are used to diagnose applications.
 - tells us what the program is doing at each step and the results of the actions.
- **Info** logs essential information of an app s.a., service start, service stop, configurations, and assumptions.
- **Warn** indicates the first level of application failure.
 - Examples: repeated access to a resource or missing secondary data.
- **Error** affects the operation result but does not terminate the program.
- **Fatal** causes termination of the program.

Testing

Log Format

- General format: [date time][log level][message]

- Example 1:

[2026-02-23 19:32:06][INFO][user “asid” had registered]

- Example 2:

[2026-02-23 19:32:06][INFO][user “mapu” cannot log in because the database is unavailable]

Explain the logs above.

Testing

Unit Testing

- popular methodology performed at development stage.
- a unit is a piece of code that does one task, e.g., a class, a procedure, or a function.

unit testing is the smallest testable part of an application.

- the key is to isolate a specific unit and check how it works.
- we don't know the operations, but we know the expected output for each set of input arguments.

How then do you think unit tests work?

- NB: No step should be called in unit tests!

Testing

Manual Vs Automated

- with every small change, retesting is essential.
- since testing is repetitive, and manual testing may be daunting.
- automating unit tests saves time and makes it easier.
- automated unit tests are reusable

Tip:

- Execute unit tests after adding new feature to see whether tyour application is affected,

Testing

Unit Testing Fundamental Logic

x = 2

y = 2

expected = 4

output = add(x, y)

if expected == output:

 print("Passed")

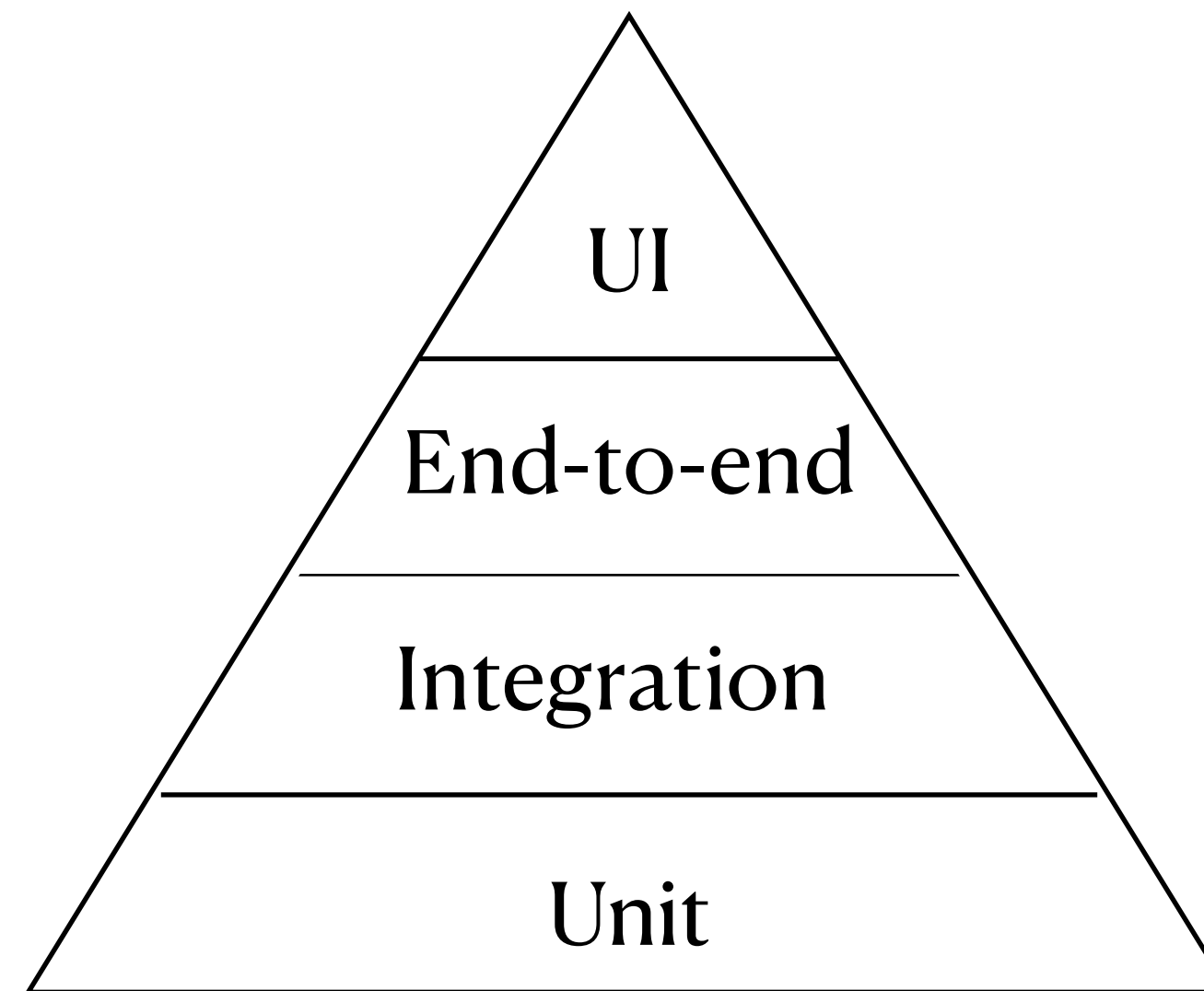
else:

 print("Failed")

Testing

Functional Testing

- checks whether a product satisfies a set of requirements, i.e., its functionality.
- usually a specification is used to clearly indicate what is acceptable.
- types of functional tests: Unit, Integration, End-to-end testing, and UI.
- test pyramid:



Testing

Test Pyramid

- **Unit Tests:** isolates individual parts of a program.
 - usually done at early stages of development.
- **Integration Tests:** software apps are usually integrated with databases, files, etc.
 - tests how all components work together.
- **End-to-end Tests:** a software methodology to test the entire application flow.
 - usually performed in real-world scenarios.
- **UI Tests:** checks the interaction with the app.
 - a.k.a. beta or acceptance testing, and is performed at the end of the project.

Testing

Self-documenting Code

```
size_t count_alphabetic_chars(const char *text)
{
    if (text == NULL)
        return 0;

    size_t count = 0;

    while (*text != '\0')
    {
        if (is_alphabetic(*text))
            count++;
        text++;
    }

    return count;
}
```

Testing

Documentation

- Inline Comments:
- Docstrings:
- High-level documentation:
- External documentation:

Testing

Further Study

- Git codelab:

<https://toolkit.codelabdavis.com/developers/overview/version-control/>

- Log format:

https://en.wikipedia.org/wiki/Common_Log_Format

- Testing frameworks

Testing

References

- Git Version Control:

<https://about.gitlab.com/topics/version-control/what-is-git-version-control/>

- Git Cheat Sheet:

<https://git-scm.com/cheat-sheet>

- Self-documenting Code:

https://en.wikipedia.org/wiki/Self-documenting_code

Testing

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na